

practical3

sort.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Sorts all elements of a 2D matrix in ascending order using a brute-force
 * nested-loop comparison approach ( $O(n^4)$  complexity).
 */
public class sort {
    public static void main(String[] args) {
        // Create a Scanner object that reads input typed on the keyboard.
        Scanner sc = new Scanner(System.in);

        // Ask the user how many rows the matrix should have and read the number.
        System.out.print("Rows: ");
        int r = sc.nextInt();

        // Ask the user how many columns the matrix should have and read the number.
        System.out.print("Columns: ");
        int c = sc.nextInt();

        // Create a 2D array (matrix) with r rows and c columns.
        int a[][] = new int[r][c];

        // Tell the user to begin typing the matrix elements.
        System.out.println("Enter elements:");
        // Outer loop: go through each row of the matrix.
        for (int i = 0; i < r; i++)
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < c; j++)
                // Store the next entered number at position [i][j].
                a[i][j] = sc.nextInt();

        // Sort all elements by comparing every pair across the matrix:
        // this brute-force approach checks every element against every other element.
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                for (int x = 0; x < r; x++) {
                    for (int y = 0; y < c; y++) {
                        // Swap if current element is smaller than the compared element:
                        // this repeatedly pushes the bigger elements towards the end.
                        if (a[i][j] < a[x][y]) {
                            // Use a temporary variable so the two values can be swapped safely.
                            int temp = a[i][j];
                            a[i][j] = a[x][y];
                            a[x][y] = temp;
                        }
                    }
                }
            }
        }

        // Print a heading before showing the sorted matrix.
        System.out.println("Sorted Matrix:");
        // Outer loop: go through each row so the sorted matrix can be printed line by line.
        for (int i = 0; i < r; i++) {
            // Inner loop: print each element of the current row followed by a space.
            for (int j = 0; j < c; j++) {
                System.out.print(a[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

```
        for (int j = 0; j < c; j++) {
            System.out.print(a[i][j] + " ");
        }
        // After finishing a row, move to the next line.
        System.out.println();
    }
    // Close the Scanner to free the system resources it uses.
    sc.close();
}
}
```

ArrayMultiplication.java

```
// Import java.util.*: a wildcard import that brings in all classes from the java.util package,
// including the Scanner class used to read input typed on the keyboard.
import java.util.*;
/**
 * Reads two matrices from the user, checks that they can be multiplied
 * (the number of columns of matrix 1 must equal the number of rows of matrix 2),
 * and if the condition holds, computes and prints their product.
 */
public class ArrayMultiplication{
    public static void main(String[] args){
        // Create a Scanner object that reads input typed on the keyboard.
        Scanner sc = new Scanner(System.in); // Initializing a new Scanner object named sc

        // Getting input from user for number of rows and columns
        // Ask the user how many rows the first matrix should have and read the number.
        System.out.print("Please enter the number of rows of matrix 1 : ");
        int rows1 = sc.nextInt();
        // Ask the user how many columns the first matrix should have and read the number.
        System.out.print("Please enter the number of cols of matrix 1 : ");
        int cols1 = sc.nextInt();
        // Ask the user how many rows the second matrix should have and read the number.
        System.out.print("Please enter the number of rows of matrix 2 : ");
        int rows2 = sc.nextInt();
        // Ask the user how many columns the second matrix should have and read the number.
        System.out.print("Please enter the number of cols of matrix 2 : ");
        int cols2 = sc.nextInt();

        // Matrix multiplication is only possible when the number of columns of the first
        // matrix equals the number of rows of the second matrix.
        if(cols1!=rows2){ // Mathematical matrix condition
            // Print an error message explaining why the matrices cannot be multiplied.
            System.out.println("ERROR : Rows of matrix 1 not equal to Columns of matrix 2");
            // Stop the program because the multiplication cannot be performed.
            return;
        }else{
            // Declaring the matrices
            // Create the first matrix with rows1 rows and cols1 columns.
            int mat1[][] = new int[rows1][cols1];
            // Create the second matrix with rows2 rows and cols2 columns.
            int mat2[][] = new int[rows2][cols2];

            // Taking input for elements of matrix 1
            // Outer loop: go through each row of the first matrix.
            for(int i=0; i<rows1; i++){
                // Inner loop: go through each column of the current row.
                for(int j=0; j<cols1; j++){
                    // Prompt the user to enter the value at position (i, j) of matrix 1.
                    System.out.print("Please enter element of matrix 1 at position : " + i + "," + j + " : ");
                    // Store the entered value in the first matrix.
                    mat1[i][j] = sc.nextInt();
                }
            }

            // Taking input for elements of matrix 2
            // Outer loop: go through each row of the second matrix.
            for(int i=0; i<rows2; i++){
                // Inner loop: go through each column of the current row.
                for(int j=0; j<cols2; j++){
                    // Prompt the user to enter the value at position (i, j) of matrix 2.
```

```

        System.out.print("Please enter element of matrix 2 at position : " + i + ", " + j + " : ");
        // Store the entered value in the second matrix.
        mat2[i][j] = sc.nextInt();
    }
}

// Create the result matrix: combining an (rows1 x cols1) with an (rows2 x cols2)
// matrix gives a result with rows1 rows and cols2 columns.
int result[][] = new int[rows1][cols2]; // Mathematical condition for resultant matrix
// Calculating matrix
// Outer loop: go over each row of the result matrix.
for(int i=0; i<rows1; i++){
    // Loop over each column of the result matrix.
    for(int j=0; j<cols2; j++){
        // Loop over the shared dimension: each result cell is a sum of products.
        for(int k=0; k<cols1; k++){
            // Add the contribution mat1[i][k] * mat2[k][j] to the result cell.
            result[i][j] += mat1[i][k] * mat2[k][j];
        }
    }
}

// Outputting the result matrix
// Outer loop: go through each row of the result matrix for printing.
for(int i=0; i<rows2; i++){
    // Inner loop: print each element of the current result row followed by a space.
    for(int j=0; j<cols2; j++){
        System.out.print(result[i][j] + " ");
    }
    // After finishing a row, move to the next line.
    System.out.println();
}

}
// End of the if-else check on whether the two matrices can be multiplied.
}
}

```

bill.java

```
// Import java.util.*: brings in the Scanner class used to read text input typed on the keyboard.
import java.util.*;

/**
 * Generates a formatted shop bill by accepting item names and prices from the
 * user and displaying them along with the total amount.
 */
public class bill {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);
        // StringBuilder builds the bill text efficiently without creating many String objects.
        StringBuilder bill = new StringBuilder();

        // Ask the user how many items they purchased so we know how many slots to create.
        System.out.print("How many items did you buy : ");
        // Read the item count as an integer.
        int count = sc.nextInt();

        // Array to hold the name of each item, sized to the item count.
        String item[] = new String[count];
        // Array to hold the price of each item, sized to the item count.
        int item_price[] = new int[count];

        // Collect item names and prices
        // Loop once per item to gather its name and cost from the user.
        for (int i = 0; i < count; i++) {
            // Ask for the name of the current item.
            System.out.print("Name of item : ");
            // Store the name in the item array.
            item[i] = sc.next();
            // Ask for the cost of the current item.
            System.out.print("What is the cost : ");
            // Store the price in the item_price array.
            item_price[i] = sc.nextInt();
        }

        // Calculate total cost
        // total accumulates the final bill amount, started at 0.
        int total = 0;
        // Loop over every item price and add it to the running total.
        for (int i = 0; i < item_price.length; i++) {
            total += item_price[i];
        }

        // Build the formatted bill string
        // Add the heading line of the bill.
        bill.append("***** SHOP BILL *****\n");
        // Loop over every item and append its name and price to the bill.
        for (int i = 0; i < item.length; i++) {
            bill.append(item[i] + " : Rs.").append(item_price[i]).append("\n");
        }
        // Add a separator line above the total.
        bill.append("-----\n");
        // Add the total amount line to finish the bill.
        bill.append("Total : Rs.").append(total);

        // Print the fully built bill to the console.
        System.out.println(bill);
    }
}
```


LowerDiag.java

```
// Import java.util.*: brings in the Scanner class used to read text input typed on the keyboard.
import java.util.*;

/**
 * Reads a matrix from the user and displays its lower triangular portion
 * (elements where the row index is greater than the column index).
 * Elements outside the lower triangle are displayed as zero.
 */
public class LowerDiag {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Ask the user how many rows the matrix should have.
        System.out.print("Please enter rows of the matrix : ");
        // Read the row count as an integer.
        int rows = sc.nextInt();
        // Ask the user how many columns the matrix should have.
        System.out.print("Please enter columns of the matrix : ");
        // Read the column count as an integer.
        int columns = sc.nextInt();

        // Create a 2D array with the given number of rows and columns to store the matrix.
        int arr[][] = new int[rows][columns];

        // Read matrix elements from the user
        // Outer loop: go through each row of the matrix.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < columns; j++) {
                // Ask the user for the element at the current row and column position.
                System.out.print("Please enter element at position : " + i + "," + j + " : ");
                // Store the typed value in the matrix.
                arr[i][j] = sc.nextInt();
            }
        }

        // Print a heading showing the matrix the user typed in.
        System.out.println("Matrix inputted : ");
        // Outer loop: print each row.
        for (int i = 0; i < rows; i++) {
            // Inner loop: print each element of the current row.
            for (int j = 0; j < columns; j++) {
                System.out.print(arr[i][j] + " ");
            }
            // Move to a new line after finishing a row.
            System.out.println();
        }

        // Display lower triangle: print element if row > col, otherwise print 0
        // Print a blank line to separate the original matrix from the lower-triangle version.
        System.out.println();
        // Outer loop: go through each row again.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < columns; j++) {
                // If the row index is greater than the column index, this element belongs to the lower triangle.
                if (i > j) {
                    // Print the actual matrix element.
                }
            }
        }
    }
}
```

```
        System.out.print(arr[i][j] + " ");
        // Otherwise the element is above the diagonal, so print 0 instead.
    } else {
        System.out.print("0 ");
    }
}
// Move to a new line after finishing a row.
System.out.println();
}
}
```

MatrixTranspose.java

```
// Import java.util.*: a wildcard import that brings in all classes from the java.util package,
// including the Scanner class used to read input typed on the keyboard.
import java.util.*;

/**
 * Reads a matrix from the user and prints both the original matrix
 * and its transpose (rows become columns and vice versa).
 */
public class MatrixTranspose {
    public static void main(String[] args) {
        // Create a Scanner object that reads input typed on the keyboard.
        Scanner sc = new Scanner(System.in);

        // Ask the user how many rows the matrix should have and read the number.
        System.out.print("Please enter rows of the matrix : ");
        int rows = sc.nextInt();
        // Ask the user how many columns the matrix should have and read the number.
        System.out.print("Please enter columns of the matrix : ");
        int columns = sc.nextInt();

        // Create a 2D array with the given number of rows and columns to store the matrix.
        int arr[][] = new int[rows][columns];

        // Outer loop: go through each row of the matrix one at a time.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < columns; j++) {
                // Prompt the user to enter the value for the current position (i, j).
                System.out.print("Please enter element at position : " + i + ", " + j + " : ");
                // Store the entered value at position [i][j] of the array.
                arr[i][j] = sc.nextInt();
            }
        }

        // Print a heading before displaying the matrix the user entered.
        System.out.println("Matrix inputted : ");
        // Outer loop: go through each row so the matrix can be printed line by line.
        for (int i = 0; i < rows; i++) {
            // Inner loop: print each element of the current row followed by a space.
            for (int j = 0; j < columns; j++) {
                System.out.print(arr[i][j] + " ");
            }
            // After finishing a row, move to the next line.
            System.out.println();
        }

        // Print a heading before displaying the transposed matrix.
        System.out.println("Transpose Matrix : ");
        // The transpose swaps rows with columns, so we print arr[j][i] instead of arr[i][j].
        for (int i = 0; i < rows; i++) {
            // Inner loop: print the element using the swapped (transposed) indexes.
            for (int j = 0; j < columns; j++) {
                System.out.print(arr[j][i] + " ");
            }
            // Move to the next line after finishing each row.
            System.out.println();
        }
    }
}
```

NameSort.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Reads a list of names from the user and sorts them alphabetically
 * using the Bubble Sort algorithm with String comparison.
 */
public class NameSort {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Prompt the user to enter how many names they want to sort.
        System.out.print("Enter number of names: ");
        // Read the count as an integer.
        int n = sc.nextInt();

        // Create an array of Strings with enough slots to hold all the names.
        String names[] = new String[n];

        // Ask the user to type in the names.
        System.out.println("Enter names:");
        // Loop once for every name slot and read each name into the array.
        for (int i = 0; i < n; i++) {
            names[i] = sc.next();
        }

        // Bubble sort: compare adjacent names and swap if out of alphabetical order
        // Outer loop: controls the number of passes over the array.
        for (int i = 0; i < n - 1; i++) {
            // Inner loop: with each pass one name settles at its final sorted position, so we can ignore it next
            time.
            for (int j = 0; j < n - 1 - i; j++) {
                // compareTo returns positive if names[j] > names[j+1]
                if (names[j].compareTo(names[j + 1]) > 0) {
                    // Save the current name in a temporary variable before overwriting it.
                    String temp = names[j];
                    // Move the alphabetically-smaller name down one position.
                    names[j] = names[j + 1];
                    // Put the larger name into the later slot.
                    names[j + 1] = temp;
                }
            }
        }

        // Tell the user the sorted list is about to be printed.
        System.out.println("Names in Alphabetical Order:");
        // Loop over the entire array and print each name in its sorted position.
        for (int i = 0; i < n; i++) {
            System.out.println(names[i]);
        }
    }
}
```

Password.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Reads a password from the user and classifies it as "Strong" or "Weak".
 * A strong password must be at least 8 characters long and contain at least
 * one uppercase letter, one lowercase letter, one digit, and one special character.
 */
public class Password {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);
        // Ask the user to type in their password.
        System.out.print("Enter Password: ");
        // Read the whole line typed by the user (so spaces inside the password are kept).
        String password = sc.nextLine();

        // Wrap the password in a StringBuilder so we can conveniently read its characters.
        StringBuilder sb = new StringBuilder(password);

        // Flags that track whether we have seen each character type, all started at false.
        boolean hasUpper = false;
        boolean hasLower = false;
        boolean hasDigit = false;
        boolean hasSpecial = false;

        // Check each character for its type
        // Loop over every character in the password.
        for (int i = 0; i < sb.length(); i++) {
            // Grab the character at position i.
            char ch = sb.charAt(i);

            // If this character is an uppercase letter, remember that we found one.
            if (Character.isUpperCase(ch)) {
                hasUpper = true;
            }
            // Otherwise, if it is a lowercase letter, remember that we found one.
            } else if (Character.isLowerCase(ch)) {
                hasLower = true;
            }
            // Otherwise, if it is a digit, remember that we found one.
            } else if (Character.isDigit(ch)) {
                hasDigit = true;
            }
            // Otherwise, if it is neither a letter nor a digit, it must be a special character.
            } else if (!Character.isLetterOrDigit(ch)) {
                hasSpecial = true;
            }
        }

        // Check whether the password length meets the minimum of 8 characters.
        boolean isLongEnough = sb.length() >= 8;

        // Password must meet all criteria to be considered strong
        // All four character types AND at least 8 characters must be present.
        if (isLongEnough && hasUpper && hasLower && hasDigit && hasSpecial) {
            System.out.println("Strong Password");
        }
        // Otherwise the password is too weak.
        } else {
            System.out.println("Weak Password");
        }
    }
}
```

```
// Close the Scanner to release the keyboard resource.  
sc.close();
```

```
}  
}
```

ProductDiag.java

```
// Import java.util.*: a wildcard import that brings in all classes from the java.util package,
// including the Scanner class used to read input typed on the keyboard.
import java.util.*;

/**
 * Reads a square matrix from the user, computes the product of all
 * elements on the main diagonal (where row index equals column index),
 * and prints the result.
 */
public class ProductDiag {
    public static void main(String[] args) {
        // Create a Scanner object that reads input typed on the keyboard.
        Scanner sc = new Scanner(System.in);

        // Ask the user how many rows the matrix should have and read the number.
        System.out.print("Please enter rows of the matrix : ");
        int rows = sc.nextInt();
        // Ask the user how many columns the matrix should have and read the number.
        System.out.print("Please enter columns of the matrix : ");
        int columns = sc.nextInt();

        // Create a 2D array with the given number of rows and columns to store the matrix.
        int arr[][] = new int[rows][columns];

        // Outer loop: go through each row of the matrix one at a time.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < columns; j++) {
                // Prompt the user to enter the value for the current position (i, j).
                System.out.print("Please enter element at position : " + i + ", " + j + " : ");
                // Store the entered value at position [i][j] of the array.
                arr[i][j] = sc.nextInt();
            }
        }

        // Print a heading before displaying the matrix the user entered.
        System.out.println("Matrix inputted : ");
        // Outer loop: go through each row so the matrix can be printed line by line.
        for (int i = 0; i < rows; i++) {
            // Inner loop: print each element of the current row followed by a space.
            for (int j = 0; j < columns; j++) {
                System.out.print(arr[i][j] + " ");
            }
            // After finishing a row, move to the next line.
            System.out.println();
        }

        // Multiply all diagonal elements (where row == col)
        // Start the product at 1 because multiplying keeps the running value unchanged (1 * x = x).
        int product = 1;
        // Outer loop: go through every row of the matrix.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through every column of the current row.
            for (int j = 0; j < columns; j++) {
                // A diagonal element is one where the row index equals the column index.
                if (i == j) {
                    // Multiply the running product by the diagonal element found.
                    product *= arr[i][j];
                }
            }
        }
    }
}
```

```
        }  
    }  
  
    // Print the final product together with an explanatory message.  
    System.out.println(product + " Is the product of all the diagonal elements");  
}  
  
}
```

Reverse.java

```
// Import StringTokenizer: splits a sentence into separate words using spaces as delimiters.
import java.util.StringTokenizer;
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Reads a sentence from the user and prints each word reversed.
 * Uses StringTokenizer to split the sentence into words and
 * StringBuilder.reverse() to reverse each word.
 */
public class Reverse {
    public static void main(String[] args) {

        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Ask the user to type in a sentence.
        System.out.print("Enter a sentence: ");
        // Read the whole line the user typed and store it in the String 'sentence'.
        String sentence = sc.nextLine();

        // Split the sentence into individual words
        // StringTokenizer breaks the sentence up, treating spaces as the word delimiters.
        StringTokenizer st = new StringTokenizer(sentence);

        // Keep looping as long as there is another word left to process.
        while (st.hasMoreTokens()) {
            // Take the next word from the tokenizer.
            String word = st.nextToken();

            // Reverse the current word using StringBuilder
            // Wrap the word in a StringBuilder so we can use its reverse() method.
            StringBuilder sb = new StringBuilder(word);
            // Reverse the word and print it, keeping it on the same line with a space.
            System.out.print(sb.reverse() + " ");
        }

        // Close the Scanner to release the keyboard resource.
        sc.close();
    }
}
```

SMS.java

```
// Import Scanner: used to read the SMS message text typed on the keyboard.
import java.util.Scanner;

/**
 * Reads an SMS message from the user and calculates how many characters
 * are used and how many remain out of the standard 160-character SMS limit.
 * Alerts the user if the message exceeds the limit.
 */
public class SMS {

    /**
     * Main method: gets an SMS from the user, counts its characters,
     * and reports how many characters are left (or how many were exceeded).
     */
    public static void main(String[] args) {

        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Prompt the user to type their SMS message.
        System.out.print("Enter your SMS: ");
        // Read the whole line typed by the user and store it in the String variable 'input'.
        String input = sc.nextLine();

        // Use StringBuilder to wrap the input so we can easily obtain its character count.
        StringBuilder sms = new StringBuilder(input);

        // Get the number of characters already used in the message.
        int charactersUsed = sms.length();
        // A standard single SMS allows up to 160 characters.
        int maxCharacters = 160;
        // Calculate how many characters are still available (can go negative if over the limit).
        int charactersLeft = maxCharacters - charactersUsed;

        // Tell the user how many characters were used in their message.
        System.out.println("Characters Used : " + charactersUsed);

        // Check whether the message fits within the 160-character limit.
        if (charactersLeft >= 0) {
            // Within the limit, so show how many characters remain available.
            System.out.println("Characters Left : " + charactersLeft);
        } else {
            // Over the limit, so negate the negative value to show the shortfall in characters.
            System.out.println("SMS limit exceeded by " + (-charactersLeft) + " characters.");
        }

        // Close the Scanner to release the keyboard resource.
        sc.close();
    }
}
```

SumEnd4.java

```
// Import java.util.*: brings in the Scanner class used to read text input typed on the keyboard.
import java.util.*;

/**
 * Reads a matrix from the user and calculates the sum of all elements
 * whose last digit (units place) is 4.
 */
public class SumEnd4 {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Ask the user how many rows the matrix should have.
        System.out.print("Please enter rows of the matrix : ");
        // Read the row count as an integer.
        int rows = sc.nextInt();
        // Ask the user how many columns the matrix should have.
        System.out.print("Please enter columns of the matrix : ");
        // Read the column count as an integer.
        int columns = sc.nextInt();

        // Create a 2D array with the given number of rows and columns to store the matrix.
        int arr[][] = new int[rows][columns];

        // Read matrix elements
        // Outer loop: go through each row of the matrix.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < columns; j++) {
                // Ask the user for the element at the current row and column position.
                System.out.print("Please enter element at position : " + i + "," + j + " : ");
                // Store the typed value in the matrix.
                arr[i][j] = sc.nextInt();
            }
        }

        // Print a heading showing the matrix the user typed in.
        System.out.println("Matrix inputted : ");
        // Outer loop: print each row.
        for (int i = 0; i < rows; i++) {
            // Inner loop: print each element of the current row.
            for (int j = 0; j < columns; j++) {
                System.out.print(arr[i][j] + " ");
            }
            // Move to a new line after finishing a row.
            System.out.println();
        }

        // Sum elements whose last digit (mod 10) equals 4
        // sum accumulates the total of the qualifying elements, started at 0.
        int sum = 0;
        // Outer loop: go through each row of the matrix.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < columns; j++) {
                // The modulo operator gives the units digit; if it is 4 the element qualifies.
                if (arr[i][j] % 10 == 4) {
                    // Add this element to the running sum.
                    sum += arr[i][j];
                }
            }
        }
    }
}
```

```
        }  
    }  
}  
  
// Print the total sum of all numbers ending in the digit 4.  
System.out.println(sum + " is the sum of all numbers ending in 4");  
}  
}
```

UpperDiag.java

```
// Import java.util.*: a wildcard import that brings in all classes from the java.util package,
// including the Scanner class used to read input typed on the keyboard.
import java.util.*;

/**
 * Reads a matrix from the user and displays its upper triangular portion
 * (elements where the row index is less than or equal to the column index).
 * Elements outside the upper triangle are displayed as zero.
 */
public class UpperDiag {
    public static void main(String[] args) {
        // Create a Scanner object that reads input typed on the keyboard.
        Scanner sc = new Scanner(System.in);

        // Ask the user how many rows the matrix should have and read the number.
        System.out.print("Please enter rows of the matrix : ");
        int rows = sc.nextInt();
        // Ask the user how many columns the matrix should have and read the number.
        System.out.print("Please enter columns of the matrix : ");
        int columns = sc.nextInt();

        // Create a 2D array with the given number of rows and columns to store the matrix.
        int arr[][] = new int[rows][columns];

        // Outer loop: go through each row of the matrix one at a time.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < columns; j++) {
                // Prompt the user to enter the value for the current position (i, j).
                System.out.print("Please enter element at position : " + i + ", " + j + " : ");
                // Store the entered value at position [i][j] of the array.
                arr[i][j] = sc.nextInt();
            }
        }

        // Print a heading before displaying the matrix the user entered.
        System.out.println("Matrix inputted : ");
        // Outer loop: go through each row so the matrix can be printed line by line.
        for (int i = 0; i < rows; i++) {
            // Inner loop: print each element of the current row followed by a space.
            for (int j = 0; j < columns; j++) {
                System.out.print(arr[i][j] + " ");
            }
            // After finishing a row, move to the next line.
            System.out.println();
        }

        // Display upper triangle: print element if row <= col, otherwise print 0
        // Print a blank line as a small separator before the upper-triangular view.
        System.out.println();
        // Outer loop: go through each row for the upper-triangular display.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < columns; j++) {
                // An upper-triangular element is one where the row index is less than
                // or equal to the column index (i <= j).
                if (i <= j) {
                    // The element belongs to the upper triangle, so print its actual value.
                    System.out.print(arr[i][j] + " ");
                } else {
                    System.out.print(0 + " ");
                }
            }
            System.out.println();
        }
    }
}
```

```
        } else {
            // The element lies below the diagonal, so print 0 instead.
            System.out.print("0 ");
        }
    }
    // Move to the next line after finishing each row.
    System.out.println();
}
}
```

UsernameGenerator.java

```
// Import the complete java.util package: provides Scanner for reading keyboard input.
import java.util.*;

/**
 * Reads a first name and last name from the user and generates a username
 * by concatenating them with a dot separator (e.g., "John.Doe").
 */
public class UsernameGenerator {

    /**
     * Main method: asks the user for their first and last name, then builds
     * and prints a username in the format FirstName.LastName.
     */
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Prompt the user to enter their first name.
        System.out.println("Please enter your first name : ");
        // Read the first token (the first name) typed by the user.
        String fname = sc.next();
        // Prompt the user to enter their last name.
        System.out.println("Please enter your last name : ");
        // Read the next token (the last name) typed by the user.
        String lname = sc.next();

        // Build the username as FirstName.LastName using StringBuilder
        StringBuilder sb = new StringBuilder(fname);
        // Append a dot and the last name to the first name, then print the final username.
        System.out.println("Your username is : " + sb.append('.') + lname));
    }
}
```

wordLetterCount.java

```
// Import java.util.*: brings in the Scanner class used to read text input typed on the keyboard.
import java.util.*;

/**
 * Reads a line of text from the user, prints each character on a separate
 * line, then counts and displays the total number of words and letters.
 */
public class wordLetterCount {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Ask the user to type in a line of text.
        System.out.println("Please enter text : ");
        // Read the whole line the user typed and store it in the String 'text'.
        String text = sc.nextLine();

        // Convert the line into a char array so we can visit each character one by one.
        char arr[] = text.toCharArray();

        // Print each character on its own line
        // Loop over the entire char array.
        for (int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
        }

        // Count words (separated by spaces) and letters (non-space chars)
        // word starts at 1 because the last word has no space after it; letter counts non-space characters.
        int word = 1, letter = 0;
        // Loop over every character again.
        for (int i = 0; i < arr.length; i++) {
            // A space marks the end of one word, so increment the word counter.
            if (arr[i] == ' ')
                word++;
            // Any character that is not a space counts as a letter.
            else
                letter++;
        }

        // Print the final word and letter counts together.
        System.out.println("Number of words = " + word + " and Number of letters = " + letter);
    }
}
```